Reflection Journal:

What is the purpose of software testing in the software development life cycle?:

The purpose of software testing in the software development life cycle, otherwise known as SDLC, is to take a programs code, and then to evaluate it, to make sure that the end product meets the business or program requirements, that it functions as it is expected to, and that it is safe to use before it is deployed for use. Software testing also allows defects to be found by testing the logic of the program. Software testing also allows developers and testers to ensure that the program meets requirements to ensure that the program does what the client has asked for. By running software tests, it allows users to also ensure that the program is secure to minimize the risk of data breaches and other dangerous vulnerabilities.

How did this project reinforce this understanding?:

By taking part and completing this project, it really allowed me to understand the importance of software testing. This project really made me understand the value of test documents, test cases as well as automated testing because it really allows one to see where the program, and why it fails. This project also highlighted why testing is so valuable as it can also sometimes expose other bugs that one does not necessarily think about when writing test cases.

Compare the three testing frameworks (MSTest, NUnit, xUnit):

When it comes to comparing MSTest, NUnit and xUnit with one another, it is clear that they have their fair share of differences, even though all 3 technically do the same thing, but with different syntax. NUnit is the oldest unit testing framework in the dotnet ecosystem. Generally when testing with NUnit, one instantiates the test class once, then you run all of your test methods within that class. It also has a very descriptive builtin assert library. After NUnit, MSTest was released by Microsoft along with Visual Studio. It was primarily released to provide a builtin first party testing solution. MSTest was also designed to integrate tightly with Microsoft Visual Studio's proprietary application lifecycle management and corporate test-reporting software. Last, in 2007, along came xUnit which was created by the authors of NUnit V2. xUnit is today considered as the modern standard. It has really clean syntax, and is extremely scalable interms of performance due to its native, highly optimized concurrent

execution. It does not have specialized assertion frameworks, and is usually used with third party libraries, like FluentAssertions like we used in this project. When taking a look at the different syntax used by the different frameworks, it is clear how it moved from strict attributes to native c# paradigms:

NUnit uses [TestFixture] as its test suite label, where MSTest uses [TestClass], and xUnit does not use any.
Also, when taking a look at the single test labels, Nunit uses [Test], MSTest uses [TestMethod], and xUnit makes use of [Fact].
The setup logic also saw some progression, as NUnit uses a dedicated method with [SetUp], MSTest uses a dedicated method with [TestInitialize], and xUnit uses a standard class constructor.

What are the advantages of each?:

- MSTest: MSTest is considered as the default unit testing framework for .NET. Its advantages are that it has a seamless integration with Visual Studio, while also being supported across multiple popular OS Platforms like MacOS, and Windows.
- NUnit: NUnit is also an extremely popular testing tool for .NET. Its main advantages are that it has fast test speeds because of parallel execution. It also offers simple data testing options, and offers an extensive assertion library.NUnit is also extremely easy to integrate with major coding tools.
- xUnit: xUnit is also an extremely popular unit testing framework, and is mostly used for c# and .NET testing. It has several advantages, like xUnit saves time by running tests at the same time, it keeps tests separated, it also works with asynchronous code using the await and async pattern. More importantly, xUnit also uses little rules, and also does not make use of complicated setup methods, instead it uses a standard class constructor.

Which would you recommend for a greenfield .NET project and why?:

When it comes to a greenfield .NET project I would definitely recommend xUnit. Not only because it is considered as the modern industry standard, but also because xUnit is fairly simple to work with because of the fact that it has clean and straight forward syntax. xUnit also runs tests in parallel, making it a lot more faster than its competitors, MSTest and NUnit.

Explain the difference between functional and non-functional testing:

When it comes to the difference between functional and non-functional testing, they are considered to be the exact opposites, because functional testing verifies what the

system does, and on the other hand non-functional testing tells us how the system does it. Functional testing is usually performed before non-functional testing. Another difference between functional vs non-functional testing is that functional testing tests whether the program meets business requirements and specifications, while non-functional testing tests whether the program meets customer expectations and technical qualities.

Give a specific example from this project for each type:

An example of a functional test that was run on this program, is when I ran testing on the Transaction module of the BEMS application, like when I tested the withdraw logic. These tests focused on whether the program works as expected when a withdraw is made.

An example of a non-functional test that was run on the BEMS program, is when we tested the Reporting Engine to see whether it takes a long time to generate a 12 month system audit log.

Describe two defects you found that surprised you. What test technique led to their discovery?:

Upon testing I found that the program allows you to enter dates that are in the future. I discovered this defect through manual testing. This defect surprised me, because even I thought that the program would not allow you to perform this action, but it did.

Another defect that was found that surprised me is the settlement fee bug, where the settlement fee is calculated with a faulty formula. This defect was discovered via automated testing. It surprised me, because it really highlights how critical a simple formula mistake can be.

What does that tell you about the value of that technique?:

First of all, discovering the future date defect via manual testing essentially told me that manual testing is still important, and should not phase away. It led me to discover that manual testing brings human creativity and judgement to software testing, which is not always easily achievable through automated testing.

Next, discovering the settlement fee formula via automated testing also highlighted to me that automated testing is still important, because it allows you to tell the system the result to expect, and the system can run the test, and tell you that it returns a value that is not correct. Automated testing allows testing to be done quickly.

What is the relationship between code coverage percentage and test quality?:

When it comes to the relationship between code coverage and test quality, they are essentially two different metrics. To start with code coverage, code coverage measures what code is covered, and runs during testing, but does not check whether the code works correctly, nor does it check whether the code meets the customer requirements. On the other hand, test quality refers to how good the code runs, and whether the code meets the customer requirements or not. However, these to metrics relate to one another as quantity vs. quality, meaning good code coverage might not necessarily mean quality code, and visa versa. Code coverage essentially acts as a code diagnostic tool that is able to identify untested code, while test quality will determine if the code will meet business requirements, and actually tests the code for bugs and flaws.

Is 100% coverage sufficient to guarantee a bug free system?:

Although code coverage is a valuable metric in software testing, in my experience, especially during this project, 100% code coverage does not guarantee a bug free system at all. 100% code coverage only proves that every single line of code in your program was covered during testing, but it does not prove that the tests that where ran was actually of any value. Another important thing to note is that one can also have 100% coverage, but none of those tests have to assert anything.

Explain with reference to your own experience in this project:

During this project for the BEMS application, achieving the required code coverage percentages stipulated in the project helped me with discovering all of the happy paths of the program, but it should be noted that the code coverage was only a good starting point, but it did not necessarily indicate that the tests I wrote were of any quality, hence why I implemented robust test design techniques. As mentioned earlier, this really highlighted that code coverage just tells one what percentage of the code was covered, and not whether the tests were quality and meaningful.

Describe how mocking improved the testability of the system:

While testing the BEMS application, mocking allowed me to achieve true isolated unit testing. By mocking, the testability of the program was drastically improved because it allowed me to replace complex, slow and otherwise unpredictable parts of the BEMS application with mocked versions of those parts, like the auth service for example. It ultimately allowed me to remove the tight connectivity between the Core of the BEMS

application and the data layer of the BEMS application. First of all, by creating test dummies of interfaces like the IAuditService and other interfaces like AccountRepository, which allowed me to have better control over the BEMS application. In the case of the BEMS application, I mocked all repositories, and services like the validation services, and most importantly, the transaction engine of the BEMS application. Mocking these services and repositories allowed me to easily simulate account lockouts, deposits and transfers.

What would have been different if you tested against the real data layer?:

Should I have not used mocking during this project, and instead used the real data layer, it would have complicated things for me. Not only would this project have changed from unit testing to integration testing. A handful of things would also have been different, like how the tests would have behaved, I would have had to clean up after every test to avoid corrupting the next test, I would also of had to manage test database instances, which also complicates things. Also, if I had to have used the real data layer, tests would also have taken longer to run, because a test suite that uses mocking runs the tests in milliseconds because it runs in memory, and it has no need to establish a database connection, and also does not have the need to query database tables, or write database logs. Furthermore, testing against the real database can cause tests to become flaky, because one test's data can interfere with another tests data state. Should I have tested against a real database, I would have had to write test setups and test cleaners for every single test I ran against the BEMS application. By mocking the data layer, it allowed me to speedily run tests against the BEMS application, without worrying about setting up database instances.

If you were a test lead on this project with a team of three junior testers, how would you have organized the work differently?

To begin with, I would have given each team member a module and a framework, for example:
For team Member 1, I would have given the MSTest Framework with the Authentication Module, for team Member 2, I would have given the NUnit Framework with the Reporting Engine, and then for team Member 3, I would have given them the xUnit framework, along with the Loan Processing module. Then, I as the team lead would have done the remaining modules for every framework. I believe this distributed workload would place less stress and pressure on the team, which would lead to a better end result, and over all a better test suite per module. As a team lead, I would also have the need to oversee

the work the 3 junior testers below me has completed to ensure that it is up to standard, and follows good testing rules.

Overall, I enjoyed this project, but it was stressful at times.